Laboratorio 2026 de Programación Imperativa

Tarea 2

Información general

Se sugiere leer detenidamente todo el texto antes de comenzar la tarea. Es muy importante que se respeten todos los requisitos solicitados en esta sección y en las siguientes. Si surgen dudas, pedimos que las formulen en el foro correspondiente a la tarea.

Individualidad

Esta tarea deberá realizarse de forma **individual**. Para todas las tareas rige el Reglamento de la Facultad de Ingeniería ante Instancias de No Individualidad en evaluaciones (lectura obligatoria).

Uso de IA

Con respecto al uso de IA, este curso se adhiere a la categoría 1: Nivel de uso reducido. Está prohibido el uso de IA para generar, total o parcialmente, la solución de las tareas de evaluación asignadas por el equipo docente. Para más información, leer Sobre el uso de herramientas basadas en IA en Programación Imperativa.

Calendario

Entrega: La entrega de la tarea puede realizarse **hasta las 12:00 horas del miércoles 18 de noviembre**. Los trabajos deberán entregarse dentro de los plazos establecidos a través del receptor habilitado para este fin. **No** se aceptarán trabajos fuera de plazo.

Re-Entrega: Solo los estudiantes que realizaron la entrega pueden hacer modificaciones y realizar una segunda entrega (*re-entrega*). La reentrega puede realizarse **hasta las 18:00 horas del viernes 20 de noviembre.**

Forma de entrega

IMPORTANTE: se debe entregar un **ÚNICO** archivo llamado **tarea2** con extensión **.pas** (**tarea2.pas**) que debe contener **únicamente** el código de los subprogramas pedidos y, eventualmente, el de los subprogramas auxiliares que se necesiten para implementarlos.

Aprobación

Para aprobar el curso, es necesario aprobar este laboratorio. Para aprobar este laboratorio, el archivo `tarea2.pas` debe compilar. En caso de entregar una tarea que compile, el equipo docente le brindará una retroalimentación con los

casos de prueba que fueron exitosos y los que no, y un análisis estático de las estructuras utilizadas. Esta retroalimentación es importante para entender los errores cometidos y mejorar como programadores. Además, será útil de cara al segundo parcial.

Archivos provistos y ejecución de pruebas

Los archivos se encuentran aquí.

Los archivos **definiciones.pas** y **utils.pas** contienen constantes, tipos y procedimientos auxiliares definidos por los docentes. El programa principal para realizar pruebas está provisto por los docentes en el archivo **principal.pas**. No se debe modificar ninguno de estos archivos, dado que no serán entregados. Para usar este programa, se debe leer y seguir las instrucciones proporcionadas en la sección Cómo ejecutar los casos de prueba de la Segunda Tarea. Los casos de prueba que se tomarán en cuenta para corregir la tarea son los que se especifican en esa sección.

Introducción

Palabras Cruzadas es una variación de los crucigramas en la que no se proporcionan como pistas las definiciones de las palabras, sino que se provee explícitamente la lista o banco de palabras a colocar. Quien juega debe completar, con las palabras del banco, los espacios disponibles en el tablero y asegurarse de que todas se usen exactamente una vez.

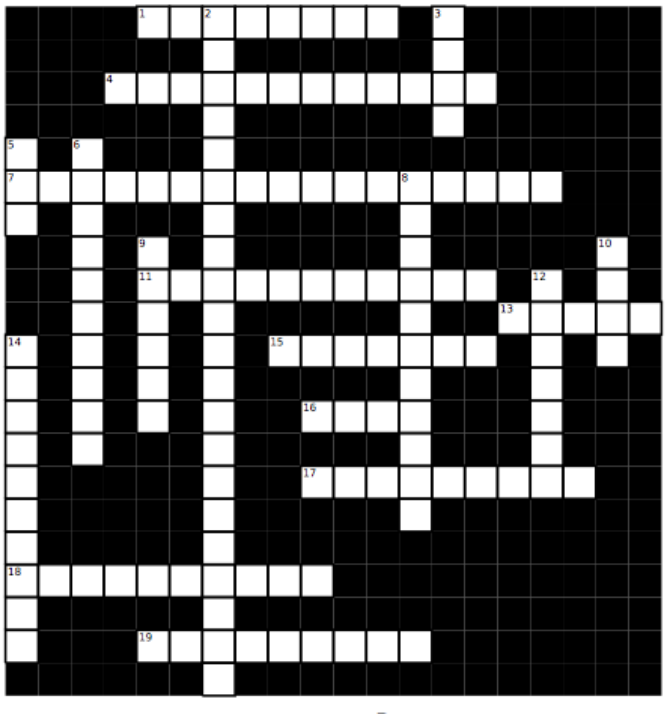


Figure 1: Un tablero de palabras cruzadas. Fuente: *Jy951950, CC BY-SA 3.0, vía Wikimedia Commons*

La tarea consiste en implementar *Pascalabras Cruzadas*, una versión de Palabras Cruzadas desarrollada en el lenguaje *Pascal*. Cada `Partida` incluye un `Tablero` con casillas habilitadas, un `Banco de Palabras` y los `Tramos` disponibles en los que se pueden insertar las palabras.

A diferencia del juego clásico, *Pascalabras Cruzadas* tiene un sistema de puntaje para poder medir el progreso durante el desarrollo del juego.

Quien juega escribe una palabra, una posición y una orientación para colocarla en el tablero. El programa debe insertar correctamente la palabra en el lugar y orientación dados.

Para colocar una palabra en el tablero, se deben respetar las siguientes reglas:

1. **Casillas bloqueadas:** las palabras no pueden ocupar ni atravesar casillas bloqueadas.
2. **Orientación:** las palabras solo pueden colocarse horizontalmente (de izquierda a derecha) o verticalmente (de arriba hacia abajo).
3. **Posición:** todas las letras de una palabra deben ubicarse en casillas consecutivas, ya sea en la misma fila o en la misma columna.

4. **Ajuste:** toda palabra, ya sea horizontal o vertical, debe encajar en un `Tramo`, es decir, en un espacio dedicado para poner palabras. La longitud de la palabra debe coincidir con la del `Tramo`.
5. **Superposición:** las palabras pueden compartir casillas con otras ya colocadas, siempre que las letras coincidan en todas las casillas compartidas.
6. **Banco de palabras:** Todas las palabras del `Tablero` deben provenir del `Banco`. Al final del juego, se obtiene el puntaje más alto cuando todas las palabras del `Banco` están en el tablero exactamente una vez (no hay repeticiones) y sin omisiones.

Archivos proporcionados

- `principal.pas`: controla la interacción.
- `utils.pas`: contiene operaciones auxiliares.
- `definiciones.pas`: define los tipos de datos.
- `tarea2.pas`: contiene los cabezales de los subprogramas, sin sus respectivas implementaciones. Este archivo es el que se debe completar.

Modelado y Tipos de Datos

Representación de una partida

TPartida El estado del juego se representa en el tipo `TPartida`, un registro que contiene un tablero (`TTablero`), un banco de las palabras a insertar (`TBancoPalabras`) y los tramos (`TTramos`) posibles en los cuales insertarlas.

Una variable de tipo `TPartida` representa la totalidad de una partida, se parte del estado inicial y sufre modificaciones a medida que el jugador realiza acciones sobre el juego (por ejemplo, insertar una palabra).

Representación del Banco de Palabras

TBancoPalabras El banco guarda todas las palabras necesarias para completar el juego, así como su estado (si están insertadas o no).

Es un arreglo con un tope que contiene hasta `MAX_CANT_PALABRAS` elementos de tipo `TPalabraInfo`.



Figure 2: Ejemplo de un `TBancoPalabras`

TPalabraInfo Representa una palabra del `Banco` y su estado en el juego. El campo palabra contiene el texto (`TPalabra`) y el campo `insertada` indica si esa palabra ya fue colocada en el tablero.

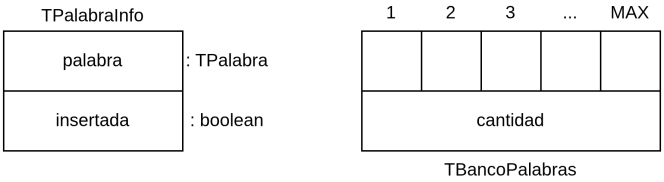


Figure 3: Diagrama de `TPalabraInfo` y `TBancoPalabras` respectivamente. Se observa que el arreglo del Banco de Palabras contiene valores de tipo `TPalabraInfo`

Representación del Tablero

TTablero Es una matriz de casillas (`TCasilla`) de tamaño `TAMANIO_TABLERO` filas y `TAMANIO_TABLERO` columnas.

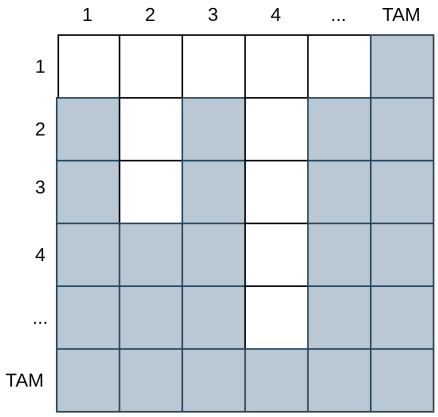


Figure 4: Diagrama de `TTablero`, las casillas claras son editables, mientras que las oscuras no

TCasilla Representa una casilla del tablero mediante un registro variante anidado.

Cuenta con dos niveles de anidación, el primer nivel es discriminado por el campo `editable`.

- Si `editable=False`, no se puede colocar una letra en la Casilla.
- Si `editable=true` esta casilla puede contener una letra. En este caso se tiene acceso al campo `escrita`, que es a su vez otro campo discriminante (de segundo nivel):
  - Si `escrita=false` significa que en esta casilla no hay ninguna palabra escrita.
  - Si `escrita=True` significa que en esta casilla hay una letra escrita y se puede acceder a los campos:
    - `letra`: indica la letra que ocupa esa casilla.
    - `multiplicidad`: representa la cantidad de palabras de las cuales forma parte esa letra.

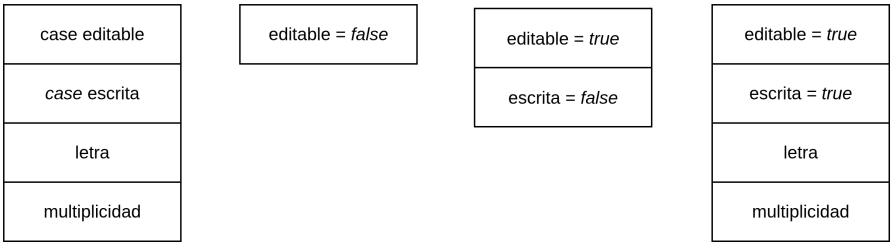


Figure 5: Esquema de `TCasilla` (izquierda) y sus tres posibles variantes

Se observa que si dos palabras se cruzan y comparten una letra, la multiplicidad de esa casilla es 2. Si se elimina una de las palabras, la letra debe permanecer porque todavía pertenece a la otra, pero la multiplicidad debe disminuir.

Representación de los Tramos

TTramos Es un arreglo con tope que almacena hasta `MAX_CANT_PALABRAS` tramos. Cada elemento describe los espacios del tablero que pueden ser ocupados (estén o no ocupados), mediante el tipo `TTramo`.

TTramo Representa un espacio del tablero en el que puede colocarse una palabra. Sus campos son:

- `orientacion`: indica si el tramo es horizontal o vertical. Es un enumerado de tipo `TOrientacion`.
- `posicion`: la ubicación de su primera casilla. Es de tipo `TPosicion`.
- `largo`: la cantidad de casillas del tramo.
- `ocupado`: si ya se colocó una palabra en ese tramo. Este, a su vez, es un campo discriminante:
  - Si `ocupado=false` significa que el tramo no está ocupado.
  - Si `ocupado=true` significa que el tramo está ocupado, en este caso se puede acceder al campo:
    - `idBanco`: es el índice de la palabra que ocupa el tramo. El índice indica en qué posición del `TBancoPalabras` de la `TPartida` se encuentra la palabra que ocupa el tramo.

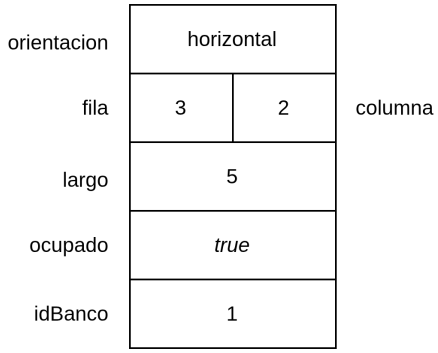


Figure 6: Ejemplo de un `TTramo`. Se observa que `ocupado = True`, por lo que `idBanco` indica que la palabra que ocupa ese tramo es el tercer elemento del Banco.

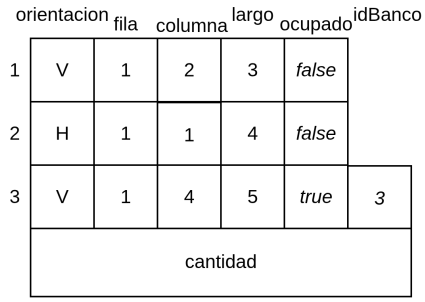


Figure 7: Eejmplo de `TTramos`.

Por ejemplo, si ocupado es `True` e idBanco es 3, el tramo contiene la palabra almacenada en `banco.arreglo[3]`.

Que un tramo esté libre no significa necesariamente que todas sus casillas estén vacías: algunas pueden contener letras de palabras que lo atraviesan.

Otros

TNatural Representa un número entero no negativo. Se utiliza para guardar cantidades, como la longitud de un texto o el número de letras que contiene una casilla.

TOrientacion Es un tipo enumerado que indica cómo se coloca una palabra. Sus posibles valores son `(horizontal, vertical)`. El primer valor indica que se escribe de izquierda a derecha, mientras que el segundo, de arriba hacia abajo.

TPalabra Arreglo con tope que permite almacenar una palabra de hasta `N` caracteres.

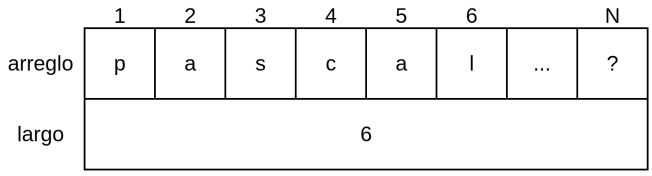


Figure 8: Ejemplo de `TPalabra`

TPosicion Registro que representa una ubicación en el casillero mediante dos valores: la fila y la columna. Los índices válidos del tablero van de `1` a `TAMANIO_TABLERO`.

En resumen

El tablero almacena las letras y el estado de cada casilla; el banco almacena las palabras disponibles y las ya insertadas; los tramos indican dónde se pueden colocar y qué palabra ocupa cada espacio.

Subprogramas a implementar

Para que el juego funcione correctamente, es necesario implementar los siguientes procedimientos:

- `sonIgualesPalabras;`
- `buscarPalabraEnBanco;`
- `insertarPalabra;`
- `buscarTramoPorIdBanco;`
- `eliminarPalabra;`
- `calcularPuntaje;`

Comparación de palabras

```
function sonIgualesPalabras(palabra1, palabra2: TPalabra): boolean;
```

Devuelve `True` si `palabra1` y `palabra2` son iguales. Es decir, tienen el mismo largo y los mismos caracteres en cada posición; devuelve `False` en caso contrario.

La implementación debe seguir un esquema Recursivo.

Sugerencia: Se recomienda seguir el enfoque utilizado en el ejercicio 8 del Práctico 12, empleando una función auxiliar que resuelva el problema de forma

recursiva y una función principal que actúe como envoltorio, invocando a la auxiliar y devolviendo su resultado.

Buscar Palabra en el Banco

```
procedure buscarPalabraEnBanco(palabra: TPalabra;
                               banco: TBancoPalabras;
                               var encontre : boolean;
                               var idBanco : integer
                              );
```

Busca si `palabra` forma parte del `banco`. Si la encuentra, devuelve `True` en `encontre` y su índice en `idBanco`. Si no la encuentra, devuelve `False` en `encontre`.

Insertar una palabra

```
procedure insertarPalabra(pos: TPosicion; ori: TOrientacion;
                          palabra: TPalabra;
                          var partida : TPartida
                         );
```

Inserta la palabra `palabra` en el tablero de la partida comenzando en `pos` y siguiendo la orientación `ori`. Actualiza las estructuras tramos y banco para reflejar la inserción realizada.

Precondiciones:

- La palabra pertenece a `partida.banco` y no está insertada.
- Existe un tramo en `partida.trams` con posición inicial `pos` y orientación `ori`.
- Ese tramo está desocupado y su longitud coincide con la de la palabra.
- Todas las casillas que ocupará la palabra están dentro del tablero y son editables.
- En cada casilla ya escrita, la letra coincide con la correspondiente de la palabra.

Se asume que estas condiciones se cumplen al invocar el procedimiento; no se requiere validarlas ni devolver un resultado que indique si la inserción fue posible.

El procedimiento debe:

- Colocar las letras en la posición correspondiente del tablero.
- Actualizar el estado de la casilla (puede haber actualizaciones de `escrita` y/o `multiplicidad`).

- Actualizar el Banco para marcar a `palabra` como `insertada`.
- Marcar el tramo correspondiente como `ocupado` y guardar en `idBanco` el índice de la palabra en el banco.

Sugerencia: Se recomienda utilizar el procedimiento `buscarTramoPorPosicion`, proporcionado por el equipo docente. En el Anexo se indican su definición y su comportamiento.

Buscar el tramo donde una palabra está insertada

```
procedure buscarTramoPorIdBanco(idBanco: integer;
                        tram: TTramos;
                        var encontre: boolean;
                        var idTramo: integer
                       );
```

Busca en `tram` el tramo ocupado cuyo campo `idBanco` coincide con el parámetro recibido. Si lo encuentra, devuelve `True` en `encontre` y el índice del tramo en `idTramo`. Si no lo encuentra, devuelve `False` en `encontre` y deja `idTramo` sin modificar.

Eliminar una palabra

```
procedure eliminarPalabra(idBanco: integer;
                          var partida : TPartida);
```

Elimina de la partida la palabra cuyo índice es `idBanco` en `partida.banco`.

Precondiciones:

- `idBanco` está entre 1 y `partida.banco.cantidad`.
- La palabra identificada por `idBanco` está insertada.
- Existe un tramo ocupado cuyo campo `idBanco` coincide con el recibido.

El procedimiento debe:

- Disminuir en 1 la multiplicidad de cada una de esas casillas. Si la multiplicidad es 0, marcar la casilla como no escrita. Si la letra aún pertenece a otra palabra, conservar su letra y su estado de `escrita`.
- Marcar el campo `insertada` del banco correspondiente a `idBanco` en `False`.
- Marcar el campo `ocupado` del tramo correspondiente a `False`.

Calcular puntaje

```
function calcularPuntaje(tablero : TTablero) : TNatural;
```

Dado un tablero, devuelve el número de casillas escritas (con letras insertadas). Cada casilla cuenta una sola vez, independientemente de su multiplicidad; por lo tanto, una letra compartida entre dos palabras aporta un único punto.

Ejemplo de ejecución

Para ayudar con la comprensión de las estructuras del juego, se presenta una ejecución completa junto con las modificaciones que sufren `partida.banco`, `partida.tablero` y `partida.tramos`:

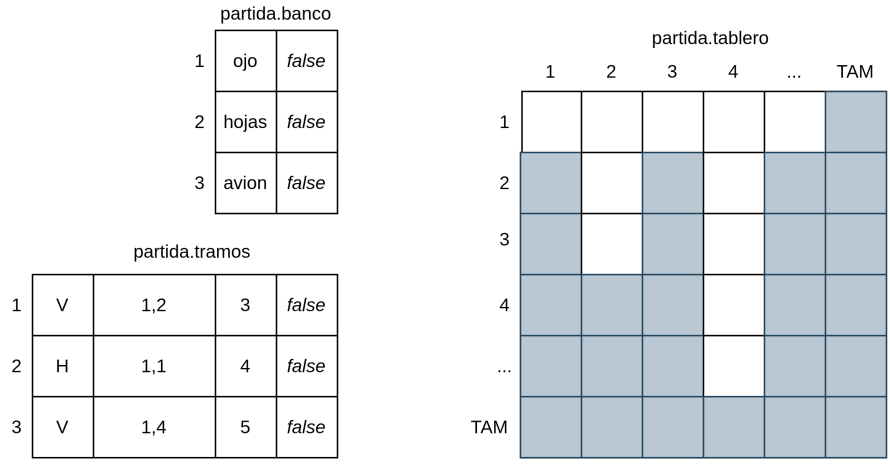


Figure 9: Se parte del estado inicial de una partida.

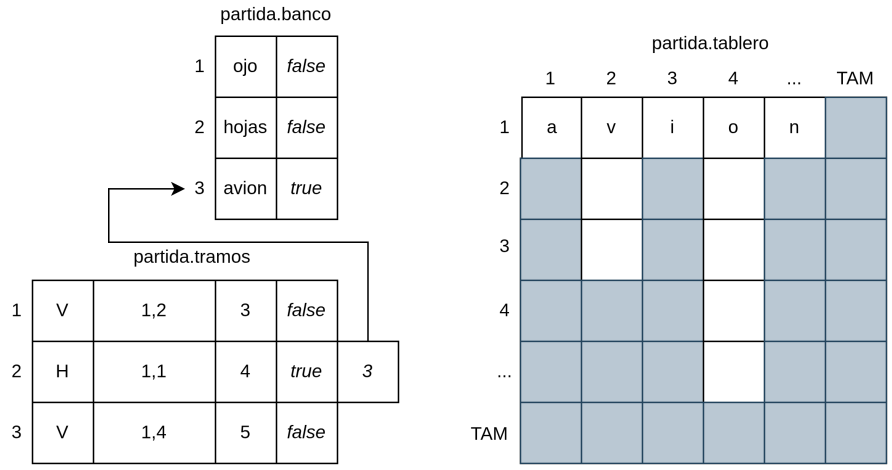


Figure 10: Se inserta la palabra *avion* con orientación *horizontal* en la celda *(1,1)*. Se ilustra con una flecha que indica que el campo `idBanco` actúa como una referencia o cursor hacia el índice del Banco correspondiente a la palabra insertada.

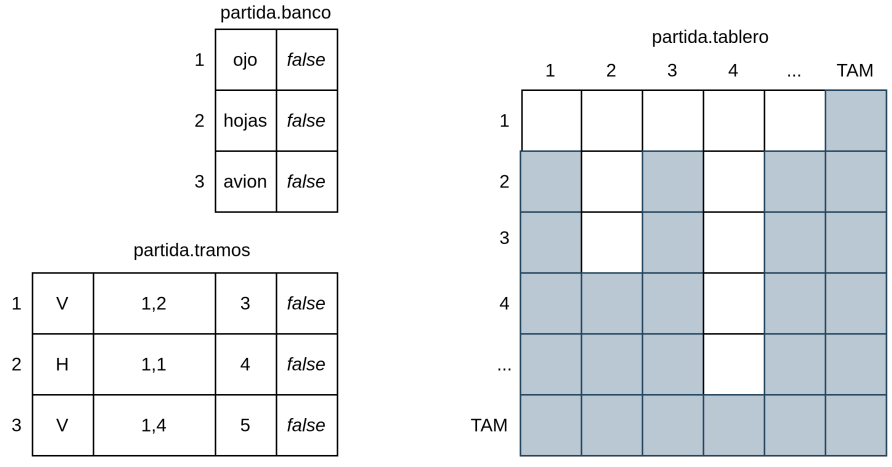


Figure 11: El jugador se da cuenta de que no puede insertar otra palabra porque no coinciden las letras en los cruces, por lo que invoca a eliminar con `idBanco` = 3. El estado del tablero queda igual que al comienzo de la partida.

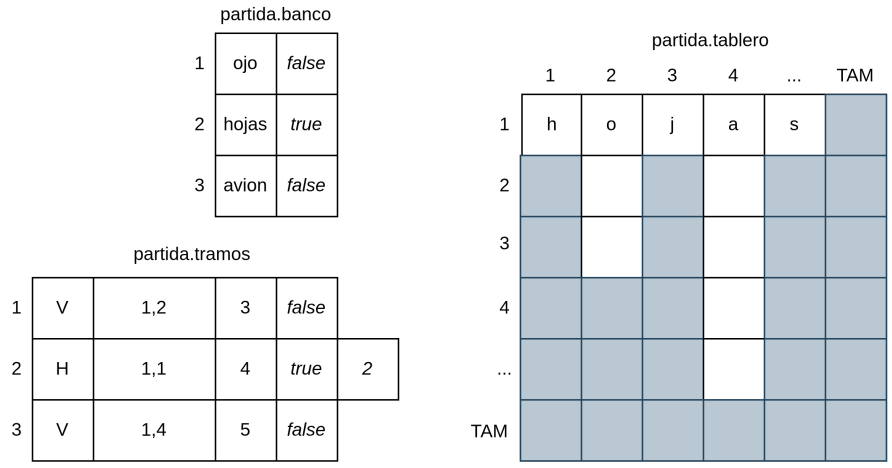


Figure 12: Ahora el jugador inserta la palabra *hojas* con orientación *horizontal* en la celda *(1,1)*.

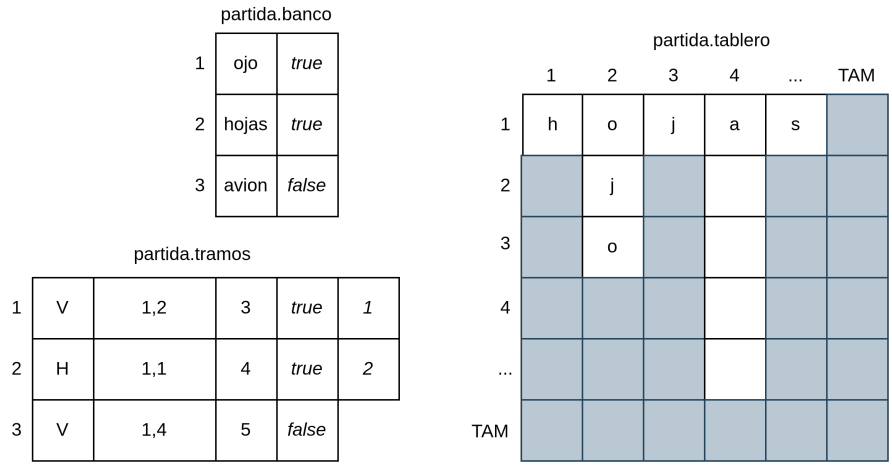


Figure 13: El jugador inserta la palabra *ojo* con orientación *vertical* en la celda *(1,2)*.

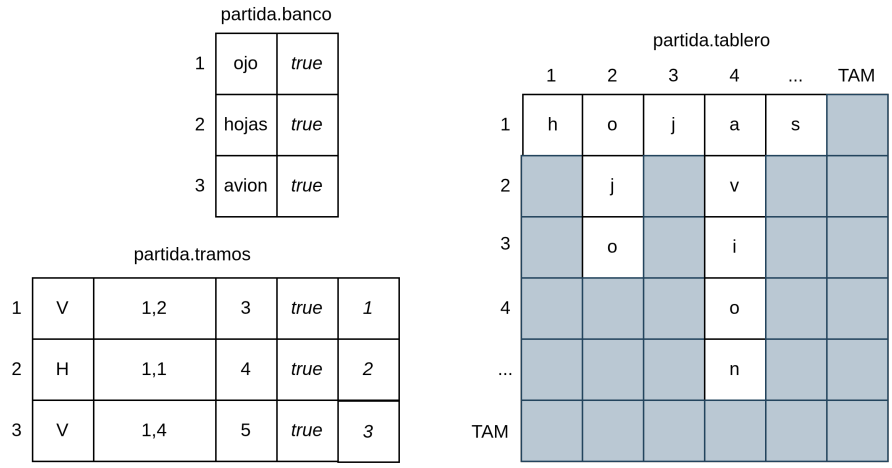


Figure 14: El jugador inserta la palabra *avion* con orientación *vertical* en la celda *(1,4)*.

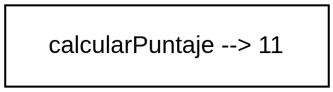


Figure 15: Por último, el usuario desea ver su puntuación, por lo que llama a *calcularPuntaje*. La función devuelve el valor **11**.

Se pide

Escribir un archivo `tarea2.pas` que contenga todos los subprogramas solicitados. Los encabezados de los subprogramas **deben coincidir exactamente** con los que figuran en esta letra. Si el estudiante realiza algún cambio, se considerará que el subprograma no se implementó. Si el estudiante lo desea, puede implementar subprogramas auxiliares adicionales (además de los solicitados).

Para la corrección, las tareas se compilarán con una versión igual o posterior a **3.0.4 para Linux**. La compilación y la ejecución se realizarán en la línea de comandos. El comando de compilación se invocará de la siguiente manera:

```
fpc -Co -Cr -Miso -gl principal.pas
```

`principal.pas` será el programa principal entregado por el equipo docente.

NO se debe compilar con el IDE de Free Pascal.

No está permitido utilizar facilidades de Free Pascal que no forman parte del estándar y que no se dan en el curso. Así por ejemplo, no se puede utilizar ninguna de las palabras siguientes: `uses`, `crlscr`, `gotoxy`, `crt`, `readkey`, `longint`, `string`, `break`, etcétera.

En esta tarea, como en todos los problemas de este curso, se valorará, además de la lógica correcta, el uso de un buen estilo de programación de acuerdo con los criterios impartidos en el curso. De esta manera, se hará énfasis en buenas prácticas de programación que lleven a un código legible, bien documentado y mantenible, tales como:

- indentación adecuada,
- utilización correcta y apropiada de las estructuras de control,
- código claro y legible,
- algoritmos razonablemente eficientes,
- utilización de comentarios que documenten y complementen el código,
- utilización de constantes simbólicas,
- nombres mnemotécnicos para variables, constantes, etcétera.

Para resolver la tarea se pueden utilizar todos los conocimientos vistos en el curso.

Anexo: Interacción

El programa `principal.pas` ofrece una interfaz de terminal. Al comenzar se elige un nivel del 1 al 5; luego se muestran el tablero, el banco de palabras y un menú para insertar palabras, eliminarlas, consultar el puntaje o salir. Para cada acción, el programa solicita los datos necesarios. Las palabras se ingresan con punto al final.

Una casilla no `editable` corresponde en la interfaz a un cuadrado relleno, mientras que una casilla `editable` y no `escrita` corresponde a un cuadrado sin relleno.

Se cuenta con un menú para probar las funcionalidades solicitadas de manera individual. Se accede mediante la opción **4. Verificar funcionalidades individualmente** del menú principal. Allí pueden probar la comparación de palabras y las búsquedas, consultar el banco, las casillas, los tramos y las multiplicidades en detalle. Estas consultas permiten revisar los efectos de las inserciones y eliminaciones. La opción **0** permite volver a la partida.

El **nivel 5** está pensado para ser modificado. Los estudiantes pueden crear sus propias Palabras Cruzadas y experimentar con ellas. Este nivel **no forma parte de la evaluación**.

Anexo: buscarTramoPorPosicion

Este procedimiento se encuentra disponible para su uso. Se recomienda su utilización para la implementación de `insertarPalabra`.

```
procedure buscarTramoPorPosicion(pos: TPosicion; ori: TOrientacion;
                                 tram: TTramos;
                                 var encontre: boolean;
                                 var idTramo: integer);
```

Este procedimiento busca el primer tramo con la posición inicial `pos` y la orientación `ori`. Si lo encuentra, devuelve `True` en `encontre` y su índice en `idTramo`; de lo contrario, devuelve `False` en `encontre` y 0 en `idTramo`.